

1 Regex: `NUMBER=r'[0-9]+'`

`IDENT=r'[A-Za-z_][A-Za-z0-9_]*'`

`WHITESPACE=r'[\t\r\n]+'`

Maximal munch is always the longest match, not the first valid stop

letter isn't split because order only breaks ties on equal

length matches 3 chars letter matches 6 so length wins regardless of order

Real lexers match the full identifier greedily first then look up the whole lexeme against a keyword table

2 Tokens for `let x = 12 + y3;;`

`("LET""let"1) ("IDENT""x"1) ("ASSIGN""="1) ("NUMBER""12"1)`  
`("PLUS""+"1) ("IDENT""y3"1) ("SEMICOLON"";"1) ("EOF""""1)`

`y3` is one IDENT token whitespace emits nothing

3 `12x = NUMBER("12") + IDENT("x")`

NUMBER stops at `x` (not a digit) no rule matches `12x` whole so both pieces obey maximal munch independently

Danger: the programmer probably meant `12 x` or `12 * x` and since it's lexically valid the typo isn't caught here it surfaces later as a vague parser error or not at all

4 Throughput is when whitespace/comments are skipped inline rather than emitted then filtered which avoids wasted allocation and a second pass over the token list  $O(n)$  vs  $O(n)+O(m)$  Multiple passes over the source are a smell: they turn a linear scan into  $O(kn)$  which kills throughput at scale